

COMPLAINT MANAGEMENT SYSTEM

Routing, API Connection, and Component Data Flow Reference Guide

1. System Architecture & Context

The Complaint Management System is built on a client-server architecture using the MERN stack. The Presentation Layer is developed with React, utilizing single-page application (SPA) routing via React Router. The Business Logic Layer is powered by Express and Node.js, presenting a RESTful API base configured at `http://localhost:5000/api`. The Data Persistence Layer utilizes MongoDB via Mongoose object modeling to define and interact with documents. The frontend and backend communicate asynchronously using Axios HTTP client promises.

2. Frontend Routing (Client-Side Navigation)

Client-side routing allows navigation within the single-page application without page reloads. This routing structure is entirely defined in the file `frontend/src/App.jsx` using react-router-dom elements (`<Routes>` and `<Route>`). Below is the mapping of paths to page components:

Route Path	React Page Component	Description / Purpose
/	Landing.jsx	Welcome landing page of the application.
/login	Login.jsx	User authentication form (Student and Admin).
/register	Register.jsx	New user signup portal.
/dashboard	Dashboard.jsx	Student dashboard displaying complaint metrics and charts.
/AdminDashboard	AdminDashboard.jsx	Administrative panel displaying system-wide analytics.

/add	Add.jsx	Grievance submission form.
/view	View.jsx	Student-facing table showing their submitted complaints and delete features.
/update/:id	Update.jsx	Editing interface for modifying pending student complaints (utilizes useParams).
/mycomplaints	MyComplaints.jsx	Consolidated view list of student grievances.
/complaintdetails/:id	ComplaintDetails.jsx	Detail page showing full timeline and feedback for a single complaint.
/admin	AdminComplaintList.jsx	Administrative grid listing all filed complaints across categories.
/admin/update/:id	StatusUpdate.jsx	Admin interface to update status (Pending, In Progress, Resolved).
/profile	Profile.jsx	User account profile update page.

3. Backend API Endpoints (Server-Side Routes)

The Express backend routes are mapped in **backend/app.js**. The entry point maps router groups and directly handles complaints core operations. User registration/login routes are routed to backend/routes/authRoutes.js, while administrative routes are handled in backend/routes/adminRoutes.js. Below is a detailed breakdown of the available API endpoints:

Method	Endpoint URL	Backend File	Description	Access
POST	/api/auth/register	authRoutes.js	Registers a new user into MongoDB.	Public
POST	/api/auth/login	authRoutes.js	Authenticates credentials, returns signed JWT &	Public

			userId.	
GET	/api/auth/profile	authRoutes.js	Retrieves active user document context.	Protected (JWT)
PUT	/api/auth/profile	authRoutes.js	Updates user name, email, and/or hashed password.	Protected (JWT)
GET	/api/admin/complaints	adminRoutes.js	Fetches filtered list of complaints.	Public (Admin)
PUT	/api/admin/complaints/:id	adminRoutes.js	Updates status value by admin user.	Public (Admin)
GET	/api/complaints	app.js	Retrieves all complaints logged in the database.	Public
GET	/api/complaints/user/:userId	app.js	Fetches complaints where createdBy matches userId parameter.	Public
GET	/api/complaints/:id	app.js	Retrieves details of a single complaint by database ID.	Public
POST	/api/complaints	app.js	Inserts a new complaint using form data.	Public
PUT	/api/complaints/:id	app.js	Updates fields of a pending complaint.	Public
DELETE	/api/complaints/:id	app.js	Deletes a pending complaint from the collection.	Public

4. Component Connection Details (Add, View, Update)

Here is the detailed flow of how the primary React pages connect to the Express endpoints and the MongoDB database:

4.1 Add Component (Add.jsx)

Allows student users to create and submit a grievance. When the submit button is triggered, the `handleSave` event handler takes the form fields (title, category, location, description) and retrieves the active user identification from `localStorage.getItem('userId')`. It submits these details via a POST request to `http://localhost:5000/api/complaints`. The Express handler in `backend/app.js` intercepting `POST /api/complaints` instantiates a new Mongoose Complaint model with the schema mapping user (or createdBy) alongside the payload text, and invokes the asynchronous model save method. Upon successful commit to the MongoDB collection, the backend returns a 201 status code, and the React component triggers client-side redirection to the `'/view'` page via `useNavigate`.

4.2 View Component (View.jsx)

Displays the active student user's filed grievances. On component mount (inside `useEffect`), the component reads the `userId` stored locally. It issues an asynchronous GET request to `http://localhost:5000/api/complaints/user/{userId}`. The backend Express server handles this request in `backend/app.js` by calling the Mongoose query `Complaint.find({ $or: [{ user: userId }, { createdBy: userId }] })` to search the complaints collection. The resulting documents are returned in a JSON array and mapped into the local React state (`complaints`), showing each item in a grid layout. Furthermore, if a complaint is 'Pending', a 'Delete' button is available. Triggering 'Delete' sends a DELETE request to `/api/complaints/{id}`, which runs `Complaint.findByIdAndDelete()` in `backend/app.js`. The frontend UI is synchronized immediately by filtering the state list.

4.3 Update Component (Update.jsx)

Allows editing of existing complaints before they undergo administrative review. When the component loads, the complaint ID is fetched from route parameters using `useParams()`. The component immediately executes a GET request to `/api/complaints/{id}`. The backend responds with the single document context via `Complaint.findById(id)`, pre-filling the React form state. When changes are submitted via the `handleUpdate` function, an asynchronous PUT request is dispatched to `/api/complaints/{id}`. The backend Express endpoint checks if the current complaint status is 'Pending' to protect reviewed data. If the check succeeds, it updates the record using the Mongoose method `Complaint.findByIdAndUpdate(id, { ...updateData, status: 'Pending' }, { new: true })`. The backend replies with status 200 and the updated payload, and the user is redirected to the `'/view'` page.